مشروع تخرج

# نظام نقل الملفات الآمن
# مع التشفير من طرف إلى طرف

## Secure File Transfer System with End-to-End Encryption

نقل ملفات مشفّر بين جهازي Windows على الشبكة المحلية — اكتشاف تلقائي، لوحة تحكم، تحقق من الهوية والسلامة

X25519 | Ed25519 | HKDF-SHA256 | AES-256-GCM

عرض تفصيلي للمشرف | github.com/blackangiliq/abbass_momal_graduate_project

# محاور العرض

## 1. المشكلة والأهداف

لماذا النقل العادي غير آمن، وما الذي يحققه النظام

## 2. المعمارية والتصميم

المكوّنات، المنافذ، البيانات المخزّنة

## 3. التشفير بالتفصيل

المصافحة، اشتقاق المفاتيح، تشفير الإطارات، SAS و TOFU

## 4. سير العمل والاكتشاف

المخططات الانسيابية للمرسل والمستلم واكتشاف الأجهزة

## 5. التطبيق والاختبارات

لوحة التحكم، تجربة الهجوم، الاختبارات الآلية، تحليل التهديدات

## 6. التنصيب والخلاصة

التشغيل على Windows، الحدود، التطوير المستقبلي

# نقل الملفات العادي على الشبكة المحلية غير آمن



## التنصّت

Eavesdropping

برنامج مثل Wireshark على نفس الـ Wi-Fi يقرأ محتوى الملف كما هو، لأن FTP و HTTP لا يشفّران البيانات.



## التلاعب

Tampering

مهاجم في المنتصف يغيّر بايتات الملف أثناء النقل، والمستلم لا يعرف أن الملف تغيّر.



## انتحال الهوية

Man-in-the-Middle

عبر ARP spoofing يتظاهر المهاجم بأنه الجهاز الآخر، فيستلم الملف بدلاً منه ثم يمرّره.

**الحل: تشفير من طرف إلى طرف + مصادقة الهوية + فحص السلامة — بحيث لا يرى أي جهاز وسيط إلا بيانات عشوائية، وأي تعديل يُكتشف فوراً.**

# أهداف المشروع

## الأهداف

**السرية:** لا يقرأ الملف إلا المستلم (AES-256-GCM)

**السلامة:** أي تعديل يُكتشف (وسم GCM + SHA-256)

**الأصالة:** التأكد من هوية الطرف الآخر (Ed25519 + SAS)

**منع الإعادة:** عدّاد ضمني داخل الـ Nonce

**السرية الأمامية:** مفاتيح X25519 جديدة لكل اتصال

**سهولة الاستخدام:** تشغيل بنقرة واحدة، اكتشاف تلقائي، واجهة عربية

**256**
bit
طول مفتاح AES لكل اتجاه

**128**
bit
مستوى الأمان لـ X25519/Ed25519

**6**
أرقام
رمز التحقق SAS — احتمال التطابق صدفة 1 من مليون

**0.03**
%
الحمل الإضافي لكل قطعة 64KB

# مقارنة مع الطرق الموجودة

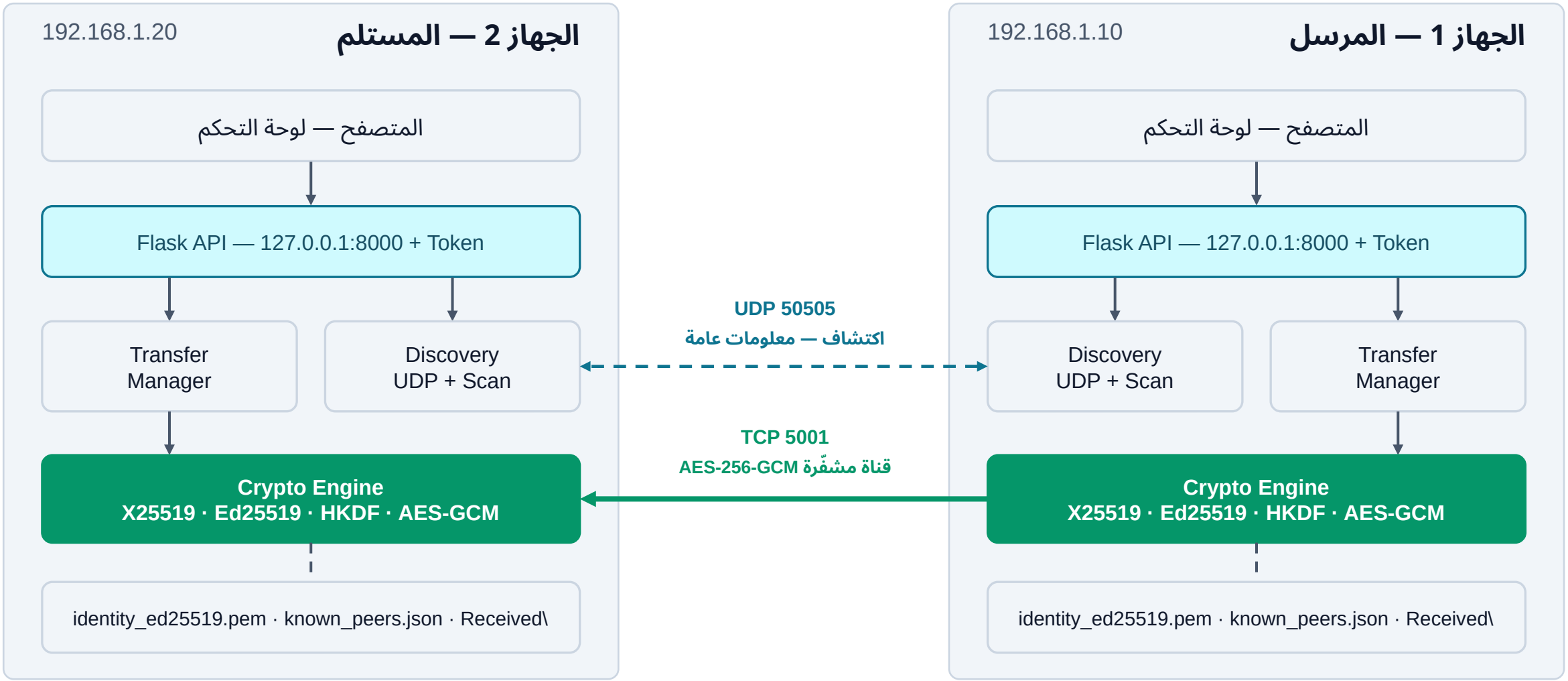
| الطريقة | تشفير | سلامة | مصادقة | سرية أمامية | اكتشاف تلقائي | سهولة الإعداد |
|---|---|---|---|---|---|---|
| مجلد مشترك SMB (افتراضي) | جزئي | جزئي | كلمة مرور | ✘ | ✔ | متوسطة |
| FTP | ✘ | ✘ | كلمة مرور بنص واضح | ✘ | ✘ | متوسطة |
| HTTP عادي | ✘ | ✘ | ✘ | ✘ | ✘ | سهلة |
| SFTP / SCP | ✔ | ✔ | ✔ | ✔ | ✘ | صعبة (خادم SSH) |
| **هذا النظام** | **✔ AES-256-GCM** | **✔ GCM + SHA-256** | **✔ Ed25519 + SAS** | **✔ X25519** | **✔ UDP + Scan** | **✔ نقرة واحدة** |

01

# المعمارية والتصميم

كيف تتوزع المكوّنات على الجهازين، وكيف تتواصل

# المخطط المعماري للنظام



# المكوّنات البرمجية والتقنيات

| الملف | الأسطر | الوظيفة |
|---|---|---|
| `crypto.py` | 342 | المصافحة، اشتقاق المفاتيح، القناة المشفّرة AES-GCM، التأطير |
| `transfer.py` | 360 | خادم الاستلام، عميل الإرسال، التقطيع، SHA-256، موافقة المستخدم |
| `discovery.py` | 209 | بث UDP، مسح الشبكة بـ 128 خيطاً، فحص IP يدوي |
| `web.py` | 151 | REST API محلي + حماية Token و Host |
| `config.py` | 84 | مفتاح الهوية، الإعدادات، قائمة الثقة TOFU |
| `netutil.py` | 88 | عناوين IP، الشبكات الفرعية، عناوين البث |
| `__main__.py` | 142 | التشغيل، عرض الـ IP، أوامر send و scan |
| `static + templates` | 485 | واجهة عربية RTL: HTML / CSS / JavaScript |

## التقنيات

**Python 3.9+**
لغة التطوير

**cryptography**
مكتبة التشفير (OpenSSL)

**Flask**
لوحة التحكم

**socket / ifaddr**
الشبكة

**pytest**
الاختبارات

**Batch + netsh**
التنصيب على Windows

# المنافذ والبيانات المخزّنة على كل جهاز

## المنافذ

| المنفذ | البروتوكول | الاستخدام | من يصل إليه |
|---|---|---|---|
| **5001** | TCP | النقل المشفّر + رد الاكتشاف | أجهزة الشبكة |
| **50505** | UDP | بث الاكتشاف | أجهزة الشبكة |
| **8000** | HTTP | لوحة التحكم | **هذا الحاسوب فقط** |

### حماية لوحة التحكم

- تستمع على 127.0.0.1 فقط
- Token عشوائي 256 بت مع كل طلب (ضد CSRF)
- فحص ترويسة Host (ضد DNS Rebinding)

## الملفات في %USERPROFILE%\SecureTransfer

| الملف | المحتوى |
|---|---|
| `identity_ed25519.pem` | المفتاح الخاص للهوية (لا يُشارك) |
| `known_peers.json` | بصمات الأجهزة الموثوقة |
| `settings.json` | الاسم، معرّف الجهاز، القبول التلقائي |
| `Received\` | الملفات المستلمة بعد التحقق |
| `tmp\*.part` | ملفات جزئية تُحذف عند الفشل |

02

# التشفير بالتفصيل

الخوارزميات، المصافحة، اشتقاق المفاتيح، تشفير البيانات

# الخوارزميات المستخدمة ولماذا اخترناها

| المرحلة | الخوارزمية | المعيار | سبب الاختيار |
|---|---|---|---|
| هوية الجهاز الدائمة | **Ed25519** | RFC 8032 | توقيع حتمي لا يحتاج رقماً عشوائياً، مفتاح 32 بايت، سريع |
| تبادل المفاتيح | **X25519 (ECDHE)** | RFC 7748 | سر مشترك بدون إرساله، مفتاح جديد لكل اتصال ⇒ سرية أمامية |
| اشتقاق المفاتيح | **HKDF-SHA256** | RFC 5869 | يحوّل السر المشترك إلى مفاتيح مستقلة موزعة بانتظام |
| تشفير البيانات | **AES-256-GCM** | NIST SP 800-38D | تشفير موثَّق AEAD: سرية + سلامة معاً، مسرّع بـ AES-NI |
| سلامة الملف | **SHA-256** | FIPS 180-4 | لا تصادمات معروفة (بعكس MD5 و SHA-1) |
| التحقق البشري | **SAS + Fingerprint** | TOFU (SSH) | كشف هجوم الوسيط عند أول اتصال |

**لماذا لا نشفّر الملف بـ RSA مباشرة؟ لأن RSA بطيء ولا يشفّر إلا بيانات صغيرة. التصميم الصحيح هو التشفير الهجين.**

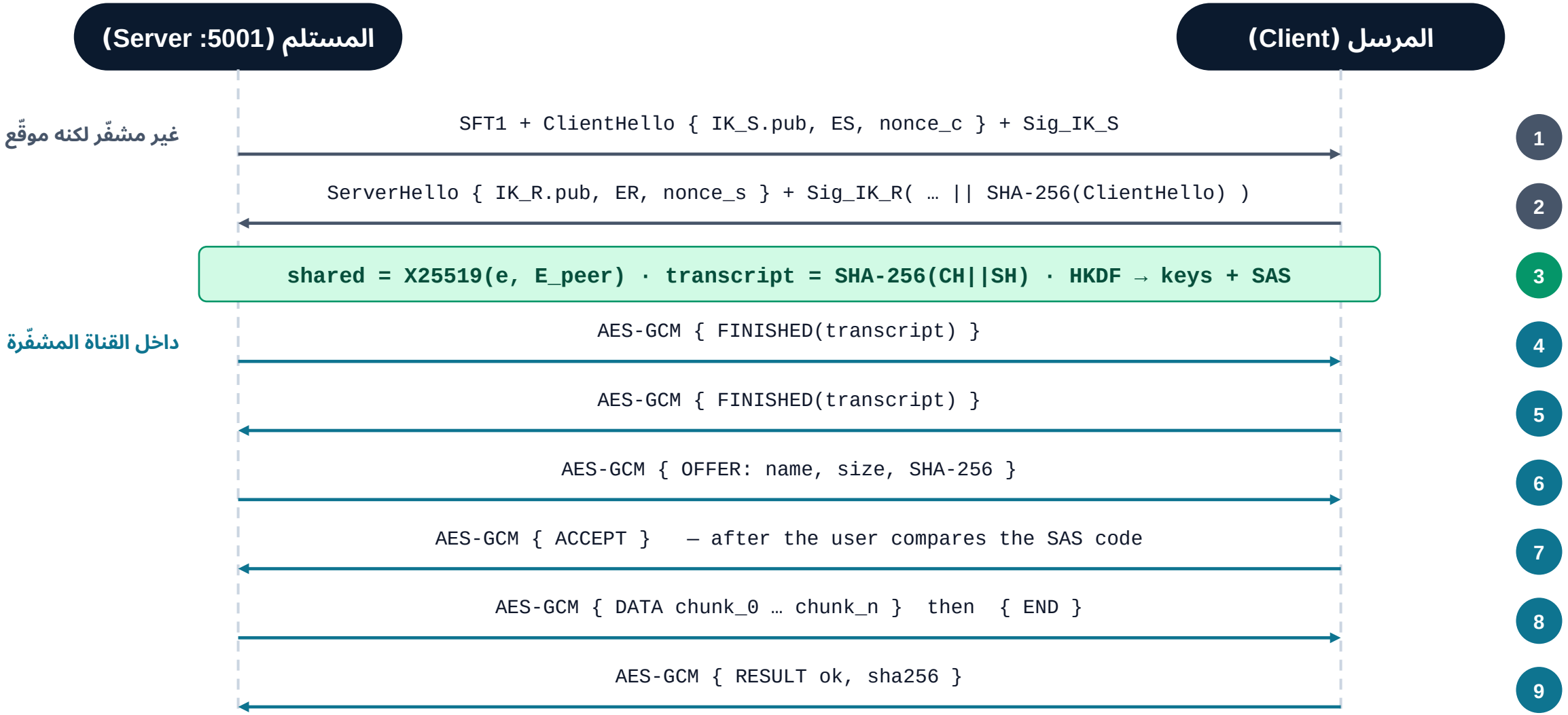
# المفاتيح في النظام

| المفتاح | النوع / الحجم | العمر | مكان التخزين |
|---|---|---|---|
| **IK — مفتاح الهوية** | `Ed25519 · 32B` | دائم لكل جهاز | `identity_ed25519.pem` |
| **e — المفتاح المؤقت** | `X25519 · 32B` | **اتصال واحد** | الذاكرة فقط |
| **shared — السر المشترك** | `32B` | **اتصال واحد** | الذاكرة فقط |
| **`K_c2s / K_s2c`** | `AES-256 · 32B each` | **اتصال واحد** | الذاكرة فقط |
| **`IV_c2s / IV_s2c`** | `4B nonce prefix` | **اتصال واحد** | الذاكرة فقط |

**بصمة الجهاز (Fingerprint) — تظهر في لوحة التحكم ويقارنها المستخدم:**

```
fingerprint = upper(hex(SHA-256("SFT-v1-FPR|" || IK_pub)))[0:32]      → 128 bit
example     = D240-E8A5-8843-E7F3-7146-D508-B015-2BB0
```

# المصافحة الآمنة خطوة بخطوة



# رياضيات المصافحة: لماذا يتطابق السر ولا يعرفه المهاجم؟

### 1) تبادل Diffie-Hellman على المنحنى (X25519)

```
ES = eS·G          ER = eR·G
Sender   : X25519(eS, ER) = eS·eR·G
Receiver: X25519(eR, ES) = eR·eS·G
          ⇒ same shared secret
```

### 2) أمان التبادل

- المهاجم يرى ES و ER فقط ولا يرى eS أو eR
- إيجاد eS من ES = مسألة ECDLP ≈ $2^{128}$ عملية
- يُرفض الناتج إذا كان كله أصفاراً (نقطة ذات رتبة منخفضة)
- المفتاحان المؤقتان يُحذفان بعد الاتصال ⇒ سرية أمامية

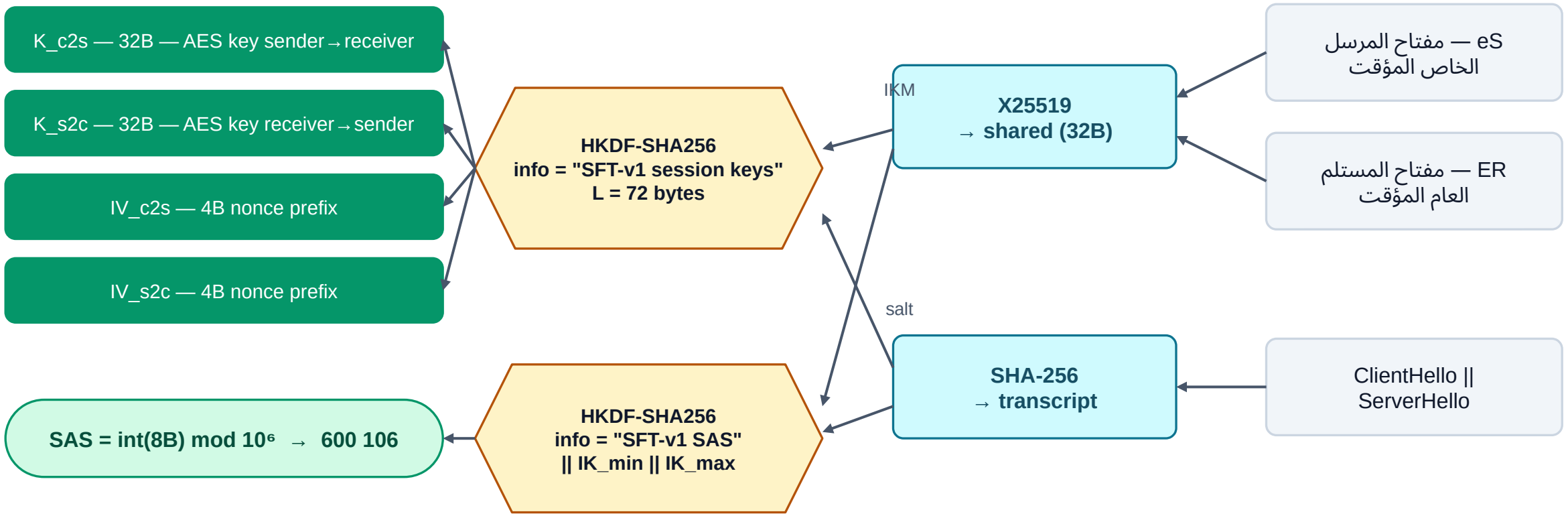
### 3) التوقيع يربط الرسائل ببعضها

```
sig_C = Sign(IK_S, "HELLO|client|" || body)
sig_S = Sign(IK_R, "HELLO|server|" || body
                   || SHA-256(ClientHello))
```

### 4) ماذا يحقق ذلك؟

- لا يمكن تغيير المفتاح المؤقت دون كسر التوقيع
- لا يمكن إعادة استخدام ServerHello قديم في جلسة جديدة
- المرسل يقارن بصمة المستلم بالبصمة المكتشفة؛ إن اختلفت يتوقف فوراً
- أي تعديل على المصافحة يغيّر الـ transcript ⇒ مفاتيح مختلفة

# اشتقاق المفاتيح بـ HKDF-SHA256



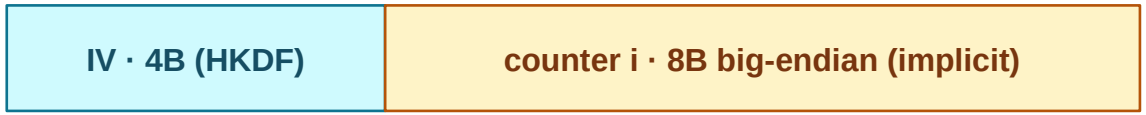

```
PRK = HMAC-SHA256(salt = transcript, shared)                    ← Extract
OKM = T(1)||T(2)||T(3),  T(i) = HMAC(PRK, T(i-1) || info || i)  ← Expand
K_c2s = OKM[0:32]  K_s2c = OKM[32:64]  IV_c2s = OKM[64:68]  IV_s2c = OKM[68:72]
```

# تشفير كل قطعة من الملف (Record Layer)

## شكل الإطار على الشبكة




### الـ Nonce (96 بت — لا يتكرر أبداً)




### البيانات الإضافية الموثّقة (AAD)

```
AAD_i = "SFT-v1|" || "c2s" || "|" || i
```

**كل هجوم على البيانات يفشل في التحقق من الوسم ⇒ قطع الاتصال وحذف الملف الجزئي**

### تعديل بت واحد

الوسم لا يطابق (احتمال التزوير ≈ $2^{-128}$)

### إعادة إطار قديم

عدّاد مختلف ⇒ Nonce مختلف

### تغيير الترتيب

العدّاد المتوقع لا يطابق

### حذف إطار

كل ما بعده يفشل

# رمز التحقق (SAS) والثقة عند أول استخدام (TOFU)

## رمز التحقق — يظهر على الشاشتين

**106 600**

- يُشتق من السر المشترك + الـ transcript + مفتاحي الهوية
- تطابق الرمزين ⇒ لا يوجد وسيط
- احتمال تطابق رمزين صدفةً: 1 من 1,000,000
- المستلم لا يقبل الملف إلا بعد المقارنة

## حالات الثقة في known_peers.json

### جديد

New

أول اتصال: قارن الرمز ثم وافق ووثّق الجهاز

### موثوق

Trusted

المفتاح مطابق للمحفوظ — يمكن تفعيل القبول التلقائي

### ⚠️ تغيّر المفتاح

Key changed

نفس الجهاز بمفتاح مختلف ⇒ المرسل يرفض والمستلم يرى تحذيراً أحمر

# بروتوكول التطبيق والتحقق من سلامة الملف

| الرسالة | الاتجاه | المحتوى |
|---|---|---|
| **OFFER** | مرسل ← مستلم | `{"name","size","sha256","chunk":65536}` |
| **ACCEPT / REJECT** | مستلم ← مرسل | قرار المستخدم (مهلة 120 ثانية) |
| **DATA** | مرسل ← مستلم | `0x02 \|\| ≤ 64 KiB` |
| **END** | مرسل ← مستلم | انتهاء البيانات |
| **RESULT** | مستلم ← مرسل | `{"ok": true, "sha256": "…"}` |

| الحد | القيمة |
|---|---|
| حجم القطعة | `64 KiB` |
| أقصى حجم إطار | `4 MiB` |
| مهلة القراءة | `30 s` |
| مهلة الموافقة | `120 s` |
| أقصى رسائل/جلسة | `2^32` |

## خطوات التحقق من السلامة

**1** المرسل يحسب SHA-256 ويضعه في OFFER المشفّر

← **2** المستلم يحدّث SHA-256 مع كل قطعة ويكتب في ملف مؤقت

← **3** بعد END: مقارنة البصمة والحجم المعلن

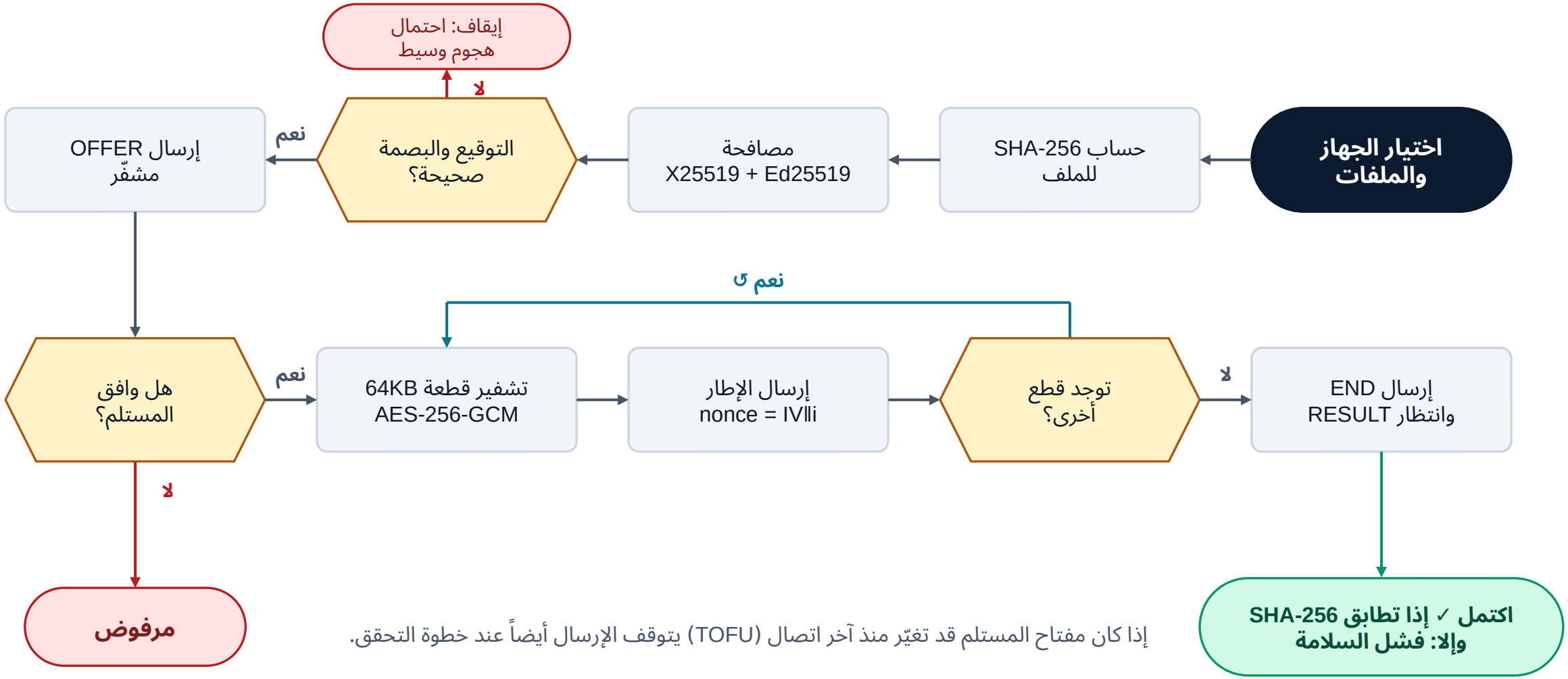
← **4** النجاح: نقل الملف إلى Received وإرسال RESULT

03

# سير العمل والاكتشاف

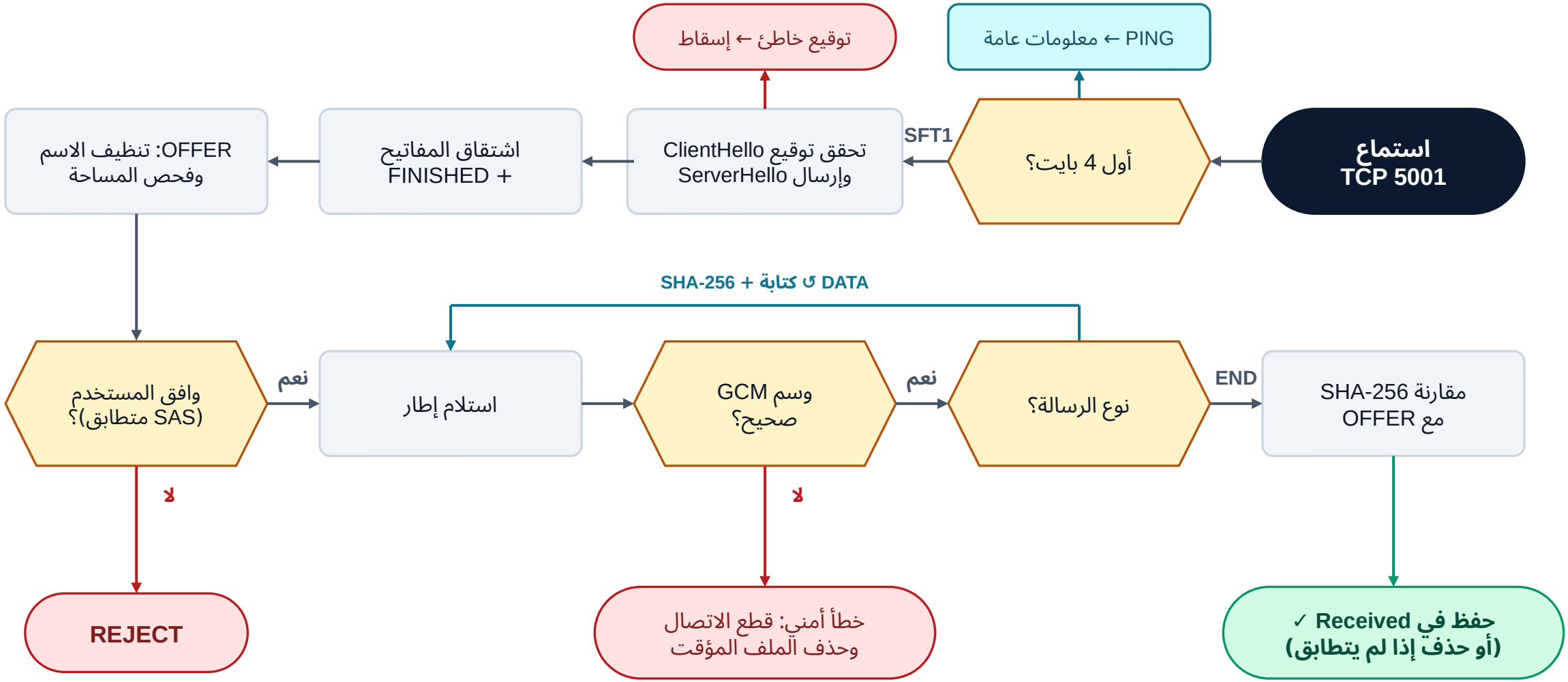
المخططات الانسيابية للمرسل والمستلم، وطرق اكتشاف الأجهزة

# المخطط الانسيابي — المرسل




إذا كان مفتاح المستلم قد تغيّر منذ آخر اتصال (TOFU) يتوقف الإرسال أيضاً عند خطوة التحقق.

# المخطط الانسيابي — المستلم



# اكتشاف الأجهزة في الشبكة المحلية — 3 طرق

## 1. بث UDP تلقائي

المنفذ 50505

- عند التشغيل: رسالة who-is-there إلى 255.255.255.255 وعنوان البث لكل شبكة
- كل جهاز يرد مباشرة
- نبضة كل 3 ثوانٍ
- غير متصل بعد 15 ثانية بلا نبضة

## 2. مسح الشبكة

TCP 5001 + PING

- زر «مسح الشبكة» + مسح تلقائي عند التشغيل
- كل عناوين /24 (253 عنواناً)
- 128 خيطاً متوازياً، مهلة 0.6 ثانية
- المدة: 1 إلى 3 ثوانٍ تقريباً

## 3. إدخال IP يدوي

IP + port

- المستخدم يكتب الـ IP الظاهر على الجهاز الآخر
- فحص PING للتأكد من وجود النظام
- رسالة خطأ واضحة عند الفشل (الجدار الناري...)
- مفيد إذا كانت الأجهزة في شبكات فرعية مختلفة

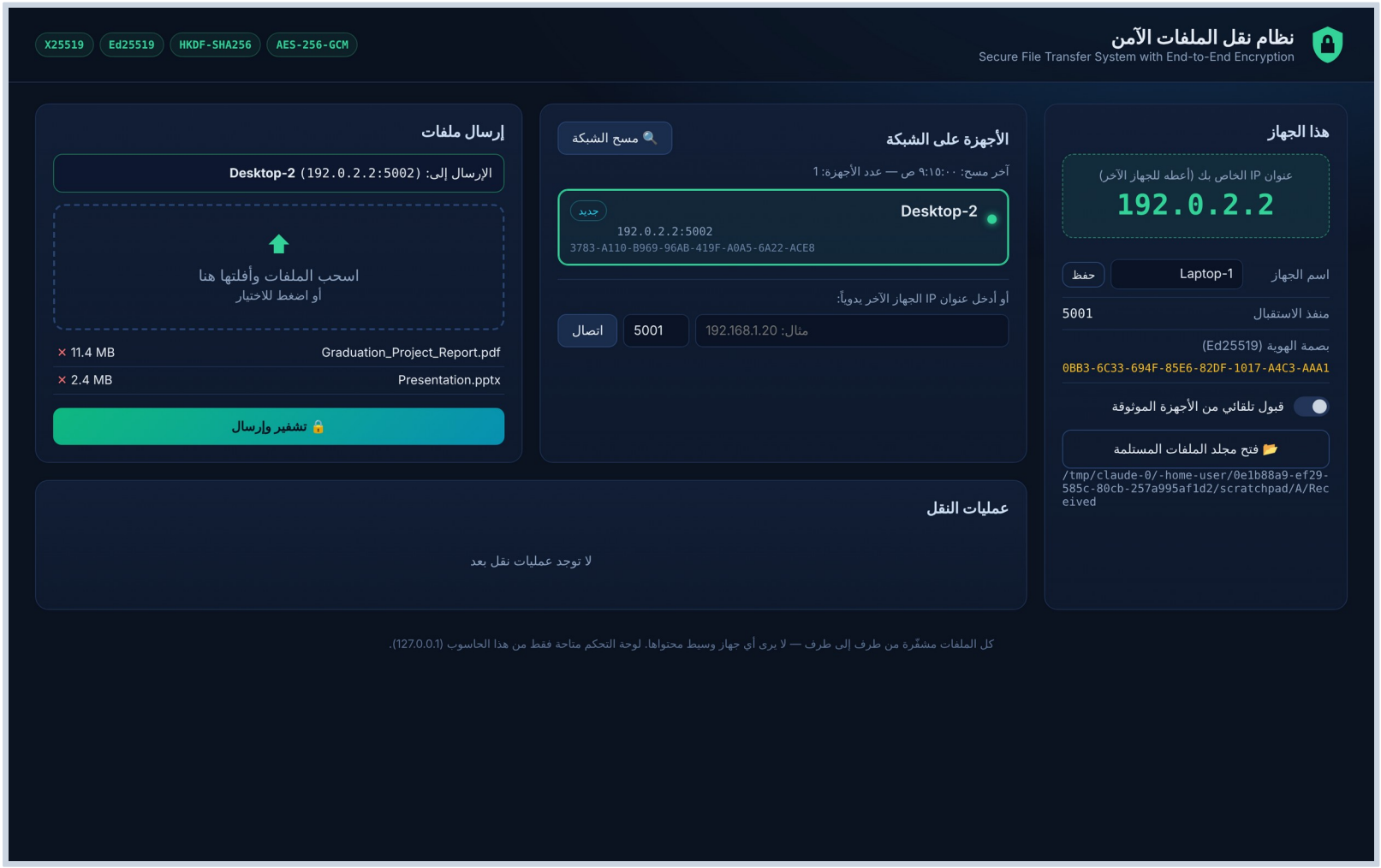
**رسائل الاكتشاف تحمل معلومات عامة فقط (الاسم، المنفذ، البصمة) وغير موثوقة. المرسل يتأكد أن المفتاح في المصافحة يطابق البصمة المكتشفة.**

04

# التطبيق والاختبارات

لوحة التحكم، تجربة الهجوم، الاختبارات الآلية، تحليل التهديدات

# لوحة التحكم — واجهة عربية




### 1 هذا الجهاز

عنوان IP بخط كبير، البصمة، الاسم، القبول التلقائي

### 2 الأجهزة على الشبكة

اكتشاف تلقائي + زر المسح + إدخال IP يدوي، مع شارة الثقة

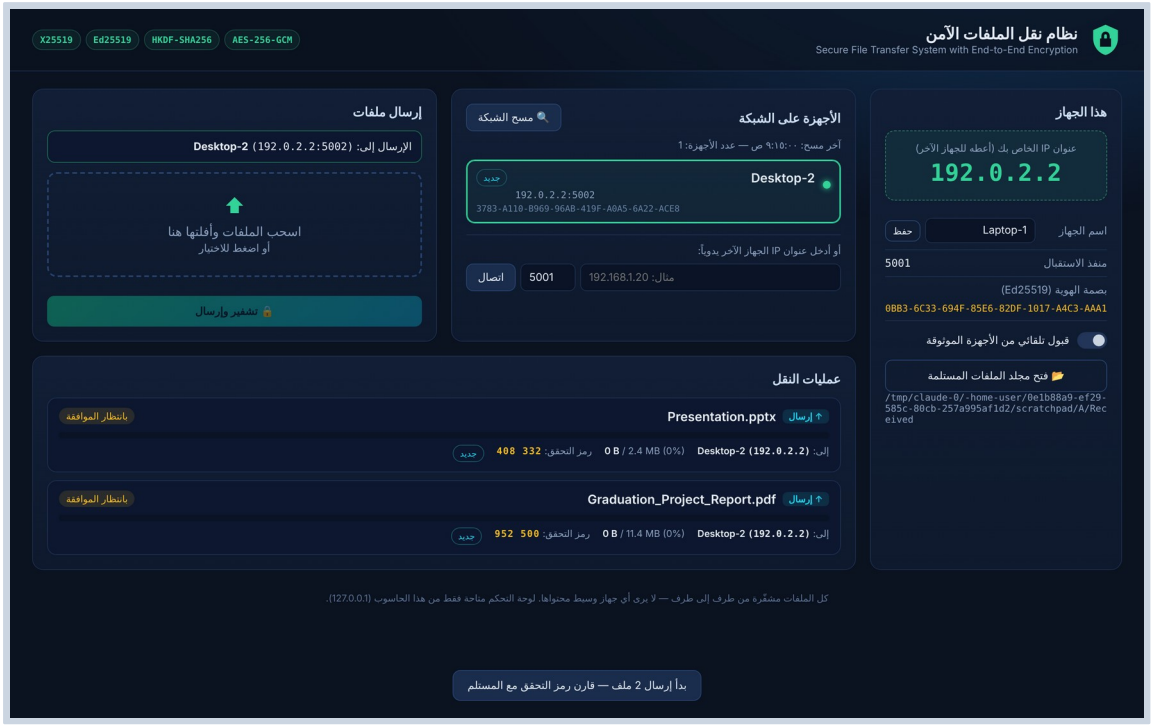
### 3 إرسال ملفات

سحب وإفلات عدة ملفات ثم «تشفير وإرسال»

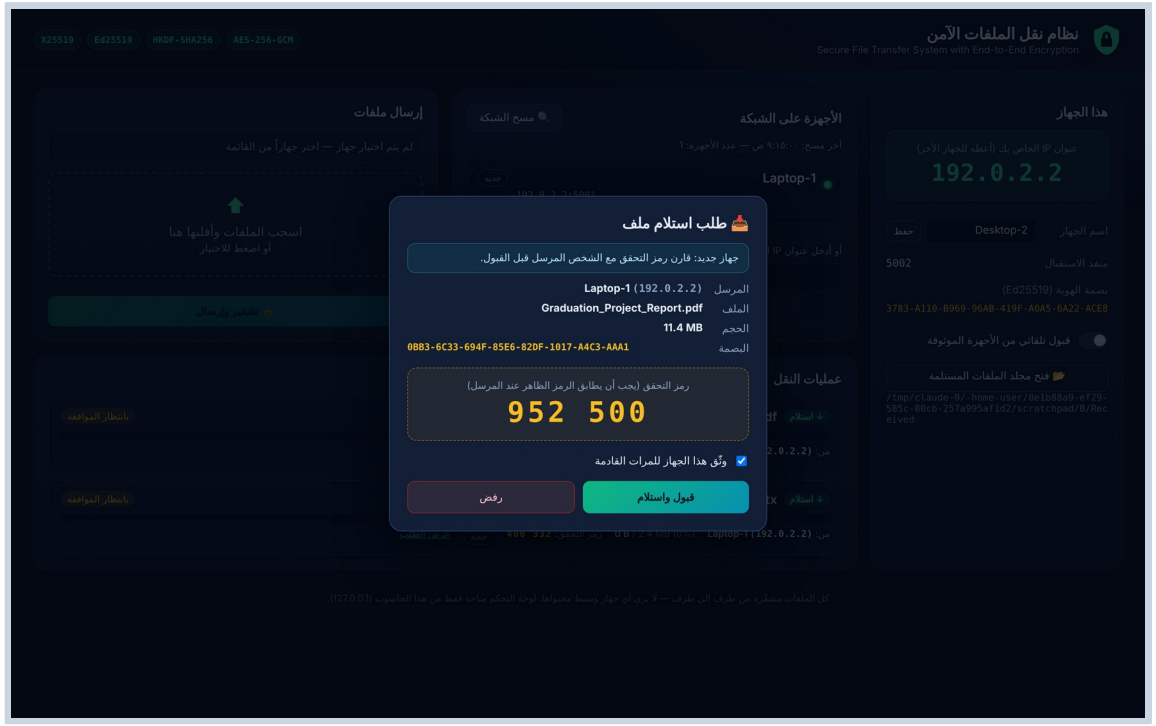
### 4 عمليات النقل

شريط تقدم، السرعة، رمز التحقق، SHA-256

# طلب الاستلام: نفس رمز التحقق على الشاشتين



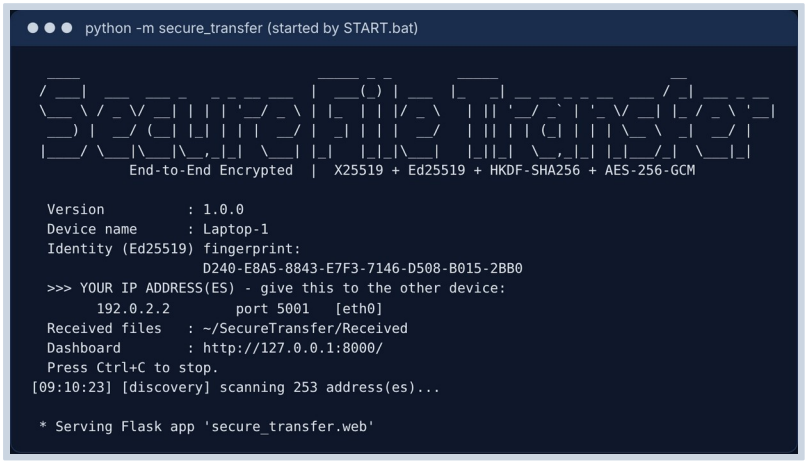

المرسل: «بانتظار الموافقة» مع نفس رمز التحقق



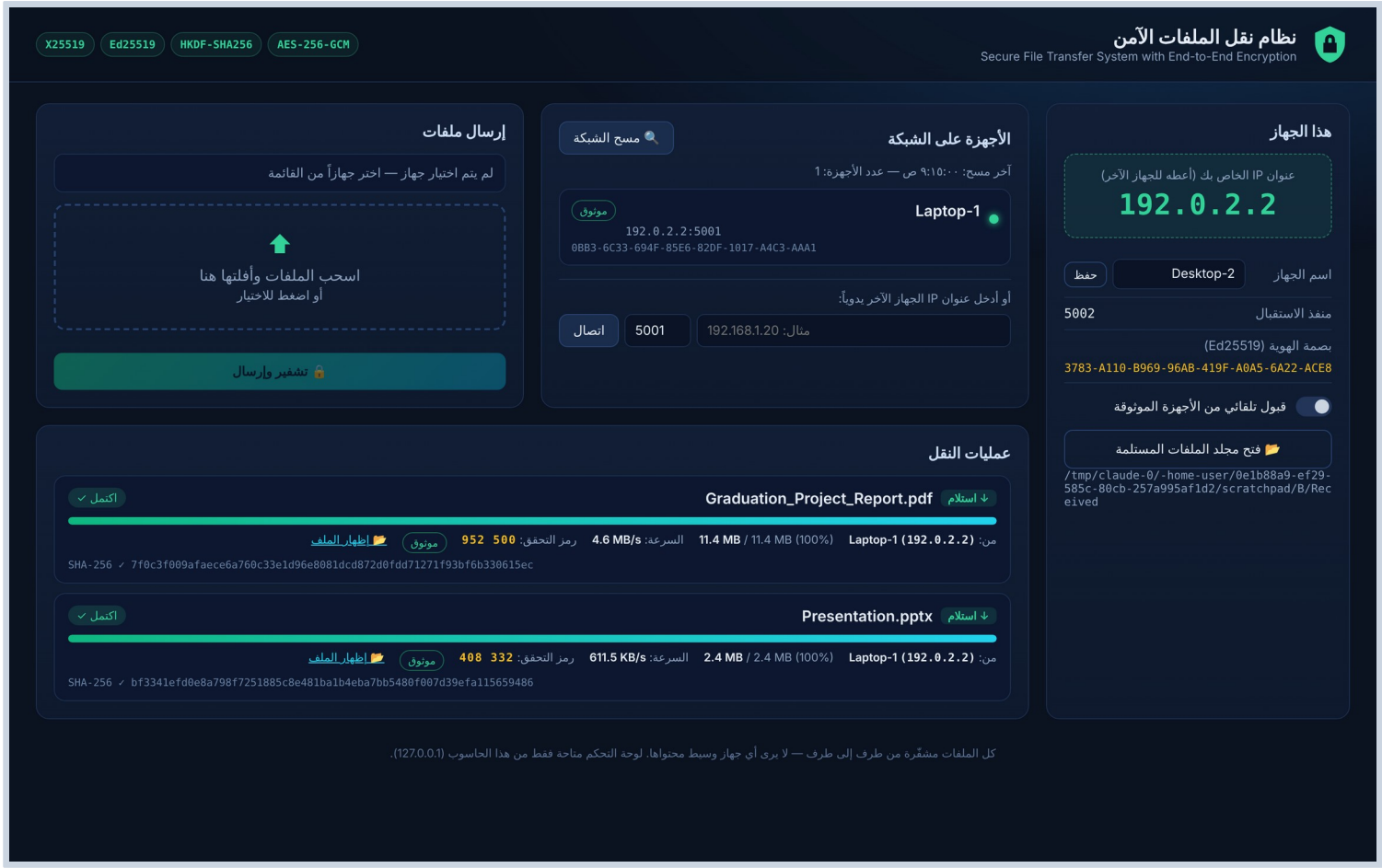

المستلم: نافذة الطلب — الاسم، الملف، الحجم، البصمة، رمز التحقق

المستخدمان يقارنان الرمز. التطابق يعني عدم وجود وسيط. عدم التطابق ⇒ رفض.

# النتيجة: اكتمال النقل مع التحقق من SHA-256




**نافذة التشغيل: الـ IP، البصمة، رابط لوحة التحكم**




**عند المستلم: «اكتمل ✓» + بصمة SHA-256 + «موثوق»**

| 100% | 4.6 | 2 |
|---|---|---|
| تطابق SHA-256 | ميغابايت/ثانية (نفس الحاسوب) | ملفات نُقلت في العرض |

# إثبات الأمان عملياً: تنصّت وتلاعب حقيقيان

## 1) التنصّت — sniff

الملف يحتوي: TOP SECRET: exam answers, password=Hunter2

```
[sender->receiver] 65536 bytes captured:
0000  00 01 00 11 f6 fb 0b f5 33 d7 7f b0
0010  8d 1b 75 20 90 5e d0 2e 61 14 b2 63
0020  96 5a 2f e0 8f ef 06 f1 fe 71 8c 2a
...   (random-looking ciphertext)
```

**0** مرة ظهرت فيها كلمة SECRET في البيانات الملتقطة. فقط المصافحة مكشوفة (مفاتيح عامة واسم الجهاز).

## 2) التلاعب — tamper

المهاجم يقلب بتاً واحداً فقط في منتصف قطعة 64KB

```
[ATTACK] flipped 1 bit in a 65536-byte segment!

receiver: [security] connection rejected:
  authentication failed: data was
  modified, replayed or forged
sender  : Result: failed
```

- الاكتشاف فوري عند أول إطار معدّل
- قطع الاتصال وحذف الملف المؤقت
- لا يُحفظ أي ملف تالف في Received

# الاختبارات الآلية (pytest)

| الاختبار | ما يثبته |
|---|---|
| `handshake_and_roundtrip` | الطرفان يشتقان نفس المفاتيح ونفس SAS |
| `wrong_expected_fingerprint` | انتحال الهوية ⇒ قطع الاتصال |
| `tampered_ciphertext` | تغيير بت واحد ⇒ فشل التحقق |
| `replayed_frame` | إعادة إطار قديم ⇒ رفض |
| `sessions_use_fresh_keys` | كل جلسة بمفاتيح جديدة (PFS) |
| `end_to_end_transfer` | نقل 3MB كامل + تحقق + حفظ الثقة |
| `reject` | الرفض لا يحفظ أي ملف |
| `safe_filename` | منع ../../Windows/System32 وأسماء محجوزة |

**8 / 8**

**اختبارات ناجحة**

```
python -m pytest -v
```

≈ 6 ثوانٍ

# تحليل التهديدات (STRIDE)

| التهديد | مثال | الحماية | النتيجة |
|---|---|---|---|
| **انتحال الهوية** | جهاز يدّعي أنه Desktop-2 | Ed25519 + مقارنة البصمة + TOFU + SAS | **يُكتشف** |
| **التلاعب** | تغيير بت أثناء النقل | وسم GCM لكل قطعة + SHA-256 | **يُكتشف** |
| **الإعادة** | إعادة قطع أو جلسة قديمة | عدّاد ضمني + مفاتيح جديدة + ربط transcript | **يُرفض** |
| **التسريب** | التنصت بـ Wireshark | AES-256-GCM؛ اسم الملف وحجمه مشفّران | **محمي** |
| **حجب الخدمة** | إطار بطول 4GB | حد 4MB + مهلات + فحص مساحة القرص | **مخفّف** |
| **`Path Traversal`** | `..\..\System32\x.dll` | تنظيف اسم الملف وأسماء Windows المحجوزة | **يُمنع** |
| **ملفات غير مرغوبة** | إرسال بدون إذن | موافقة المستخدم إلزامية | **يُمنع** |
| **CSRF** | موقع خبيث يستدعي 127.0.0.1 | Token + فحص Host + localhost فقط | **يُمنع** |
| **سرقة المفاتيح لاحقاً** | سرقة اللابتوب بعد شهر | X25519 مؤقت ⇒ سرية أمامية | **الجلسات السابقة آمنة** |

05

# التنصيب والخلاصة

التشغيل على جهازي Windows، حدود التصميم، والتطوير المستقبلي

# التنصيب والتشغيل على جهازي Windows

**1** **تحميل المشروع**

Download ZIP من GitHub وفك الضغط (أو git clone)

**2** **نقرة مزدوجة على START.bat**

يبحث عن Python 3.9+ ويثبته بـ winget إن لم يجده

**3** **تجهيز تلقائي (أول مرة)**

بيئة venv. + تثبيت cryptography و flask و ifaddr

**4** **الموافقة على صلاحية المسؤول**

قاعدتا جدار ناري: TCP 5001 و UDP 50505

**5** **التشغيل**

تظهر نافذة فيها الـ IP وتُفتح لوحة التحكم 127.0.0.1:8000

ملاحظة: شبكات Wi-Fi العامة قد تمنع الأجهزة من رؤية بعضها (Client Isolation). الحل: Hotspot من الهاتف أو راوتر منزلي.

## طرق تشغيل أخرى

```
# manual
pip install -r requirements.txt
python -m secure_transfer

# CLI send / scan
python -m secure_transfer send
   192.168.1.20 report.pdf
python -m secure_transfer scan

# single EXE (no Python)
build_exe.bat
→ dist\SecureTransfer.exe
```

# حدود التصميم والتطوير المستقبلي

## حدود حالية

- اسم الجهاز ومفاتيحه العامة مكشوفة في المصافحة والاكتشاف (اسم الملف ومحتواه مشفّران)
- حجم الملف التقريبي يمكن تقديره من حجم البيانات
- إذا تجاهل المستخدم مقارنة SAS عند أول اتصال قد ينجح مهاجم نشط (مثل SSH)
- المفتاح الخاص محفوظ بدون كلمة مرور
- لا يحمي من جهاز مخترق أصلاً
- لم يُختبر START.bat بعد على جهازي Windows حقيقيين

## تطوير مستقبلي

- حماية المفتاح الخاص بـ Windows DPAPI أو كلمة مرور (Argon2)
- تشفير هوية الجهاز في المصافحة (نمط Noise XX)
- استئناف النقل بعد الانقطاع (Merkle tree)
- إرسال مجلدات كاملة
- مقاومة الحاسوب الكمّي: X25519 + ML-KEM (FIPS 203)
- تطبيق للهاتف المحمول

# الخلاصة

### تشفير من طرف إلى طرف

AES-256-GCM بمفاتيح X25519 مؤقتة لكل اتصال

### هوية موثوقة

توقيع Ed25519 + بصمة + رمز SAS + TOFU

### سلامة مضمونة

وسم GCM لكل قطعة + SHA-256 للملف كاملاً

### سهل الاستخدام

اكتشاف تلقائي، لوحة عربية، تشغيل بنقرة واحدة

## شكراً لكم

github.com/blackangiliq/abbass_momal_graduate_project